

RAG

Retrieval Augmented Generation — A Learn-From-Scratch Guide

Concepts · Illustrated Diagrams · Build Steps · Curated Video Tutorials

Personal study guide · compiled for self-paced learning

1. What Is RAG, and Why It Exists

RAG (Retrieval Augmented Generation) is a technique that improves what a large language model (LLM) says by giving it real, relevant information at the moment you ask a question — instead of relying only on what it memorized during training.

The two problems RAG solves

Problem	Why it happens	How RAG fixes it
Hallucination on unknown or recent info	LLMs only know data up to their training cutoff. Asked about anything after that, they often generate a confident-sounding but wrong answer rather than admit they don't know.	Feed the model real, current context retrieved from a knowledge base at query time, so it answers from facts instead of guessing.
No access to private or internal data	Company documents, internal wikis, product manuals — none of this was ever in the model's training data.	Store this data externally and retrieve the relevant pieces on demand. No retraining needed, and it updates instantly when documents change.

Why not just fine-tune the model on your data instead? Fine-tuning is expensive, slow, and has to be repeated every time your underlying data changes. RAG is far cheaper to keep up to date — you just add or edit documents in your knowledge base.

The big picture

Every RAG system, no matter how advanced, is built from two pipelines wired together. Memorize this — it is the skeleton of every RAG project you will ever build.

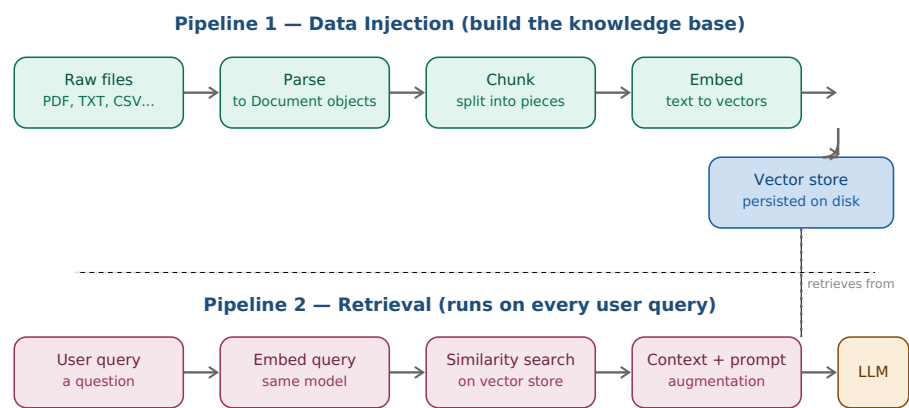


Figure 1. The two core pipelines every RAG system is built from.

2. Key Building Blocks

Concept	What it does	Common tools
Document structure	Standard container for text + metadata so every downstream step works the same regardless of source format.	LangChain <code>Document</code> (<code>page_content</code> , <code>metadata</code>)
Loaders	Convert raw files (PDF, TXT, CSV, HTML, SQL) into <code>Document</code> objects.	<code>PyPDFLoader</code> , <code>PyMuPDFLoader</code> , <code>TextLoader</code> , <code>CSVLoader</code> , <code>DirectoryLoader</code>
Chunking	Splits large documents into smaller pieces that fit inside model context limits.	<code>RecursiveCharacterTextSplitter</code>
Embeddings	Converts text into numeric vectors so meaning-similarity can be computed mathematically.	Open-source: <code>sentence-transformers</code> (<code>all-MiniLM-L6-v2</code>). Paid: OpenAI, Gemini embeddings
Vector store	Persists embeddings and metadata; supports fast similarity search.	Chroma, FAISS, Pinecone, Weaviate, Qdrant
Retriever	Takes a query, embeds it, searches the vector store, returns the top matching chunks.	Custom retriever class, or the vector store's built-in query method
Augmentation	Combines retrieved context with a prompt template before sending it to the LLM.	Simple f-string / <code>PromptTemplate</code>
Generation	The LLM produces the final answer using the augmented prompt.	Groq, OpenAI, Gemini, or any hosted / local LLM

Why metadata matters

Storing things like `source` , `page_number` , `author` , and `date` alongside each chunk lets you later filter searches (for example, "only search documents by this author") and lets you cite sources in your final answer.

Why chunking matters

Embedding models and LLMs have a fixed context window — you cannot hand a 100-page PDF straight to an embedding model. `chunk_size` controls piece length; `chunk_overlap` keeps a bit of shared text between adjacent chunks so meaning is not lost at the boundaries.

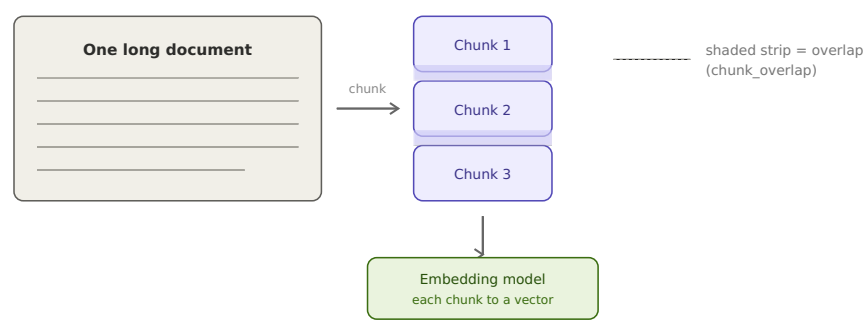


Figure 2. Chunking a document with overlap, then embedding each chunk.

3. How Retrieval Actually Works

The mental model for the math

- Embedding = text is turned into a vector (for example, 384 numbers) that captures its meaning.
- Similar meaning → vectors point in similar directions in that space.
- Similarity search (commonly **cosine similarity**) finds which stored vectors are "closest" to the query vector.
- `similarity_score = 1 - distance` is a common conversion when your vector database returns a distance metric.
- A `score_threshold` lets you discard weak or irrelevant matches instead of always returning the top results regardless of quality.

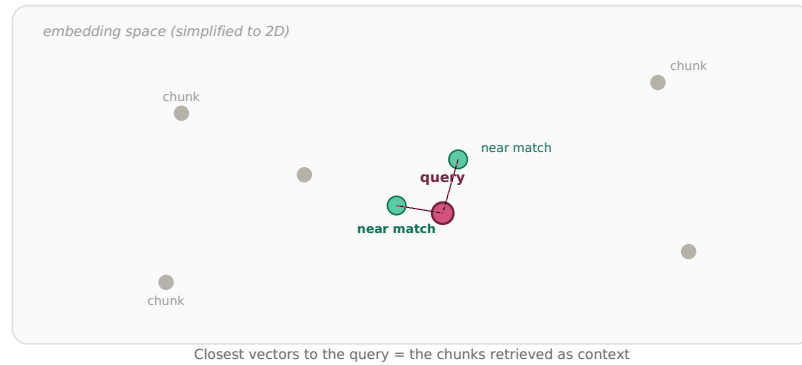


Figure 3. Similarity search finds the chunks whose vectors sit closest to the query vector.

Retrieval, step by step

1. User asks a question.
2. The question is embedded with the **same embedding model** used for the documents.
3. The vector store compares the query vector to every stored chunk vector.
4. The top-K closest chunks (above the score threshold) are returned as **context**.
5. Context + the original question are combined into a prompt.
6. The LLM generates an answer grounded in that context.

4. Build Your Own RAG Project — Step by Step

Use this as your actual build order. Each phase should be a working, testable milestone before you move to the next.

Step 1 Project setup

- Create a virtual environment (`uv` , `venv` , or `conda`).
- Install: `langchain` , `langchain-core` , `langchain-community` , `pypdf` / `pymupdf` , `sentence-transformers` , `chromadb` (or `faiss-cpu`), plus your chosen LLM SDK.
- Folder layout: `data/` , `notebook/` , `src/` (with `data_loader.py` , `embedding.py` , `vector_store.py` , `search.py`), `app.py` , `requirements.txt` .

Step 2 Data loading

Write a function that reads every file in `data/` and returns a list of `Document` objects.

```
def load_all_documents(data_directory: str) -> list:
    documents = []
    for pdf_file in Path(data_directory).glob("**/*.pdf"):
        documents.extend(PyPDFLoader(str(pdf_file)).load())
    return documents
```

Milestone: print `len(documents)` and confirm content and metadata look correct.

Step 3 Chunking

```
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(documents)
```

Milestone: confirm the chunk count is reasonable (for example, 64 documents becoming roughly 350 chunks).

Step 4 Embeddings

```
model = SentenceTransformer("all-MiniLM-L6-v2")
texts = [c.page_content for c in chunks]
embeddings = model.encode(texts, show_progress_bar=True)
```

Milestone: confirm the embedding shape, for example `(num_chunks, 384)` .

Step 5 Vector store

```
client = chromadb.PersistentClient(path="vector_store")
collection = client.get_or_create_collection(name="my_docs")
collection.add(ids=ids, embeddings=embeddings, documents=texts, metadatas=metadatas)
```

Milestone: reload the persisted store in a new session and confirm the document count matches.

Step 6 Retrieval

```
def retrieve(query, top_k=3, score_threshold=0.0):
    query_embedding = model.encode([query])
    results = collection.query(query_embeddings=query_embedding, n_results=top_k)
    # zip ids/documents/metadatas/distances, filter by score_threshold
```

Milestone: run a few test queries and manually verify the retrieved chunks are actually relevant.

Step 7 Augmented generation

```
def rag_answer(query, top_k=3):
    results = retrieve(query, top_k)
    context = "\n\n".join(r["content"] for r in results)
    if not context:
        return "No relevant context found."
    prompt = f"Use the following context to answer concisely.\n\nContext:\n{context}\n\nQuestion: {query}"
    return llm.invoke(prompt).content
```

Milestone: ask a question you know the answer exists for in your data and confirm the answer is grounded, not hallucinated.

Step 8 Level up: advanced features

Once the basic loop works end to end, layer these in one at a time:

- Return sources and a confidence score alongside the answer (source file, page number, similarity score).
- Persistence check — only rebuild the vector store if it does not already exist, otherwise load it.
- Conversation history for multi-turn chat.
- Streaming responses for better user experience.
- Better chunking strategies — semantic chunking, sentence-aware splitting.
- Query rewriting or multi-query retrieval to improve recall.
- Reranking with a cross-encoder for better precision.
- Agentic RAG — letting the model decide when and whether to retrieve, or which tool to call.

5. Common Pitfalls

- **Forgetting metadata** — you lose the ability to filter or cite sources later. Always attach `source` , `page` , and similar fields at load time.
- **Chunking too large or too small** — too large risks exceeding context limits and diluting relevance; too small loses context. Start with `chunk_size=1000` , `chunk_overlap=200` and tune from there.
- **No score threshold** — the retriever always returns something, even if nothing is actually relevant. Add a threshold and a clear fallback message.
- **Rebuilding the vector store every run** — wastes time and compute. Persist it and load on subsequent runs.
- **Treating RAG as "hallucination-proof"** — it only reduces hallucination for questions covered by your knowledge base. Outside that scope, the LLM can still hallucinate.

Glossary

Term	Meaning
Chunk	A smaller piece of a larger document, sized to fit model context limits.
Embedding	A numeric vector representation of text capturing semantic meaning.
Vector store / DB	A database optimized for storing and searching embeddings.
Retriever	The component that takes a query and returns relevant chunks.
Augmentation	Combining retrieved context with a prompt before sending it to the LLM.
Similarity score	A measure of how close two vectors are; used to rank and filter results.
Agentic RAG	A pattern where the model decides when to retrieve, which tool to use, and how many steps to take.

6. Learning Path — If You're Starting From Zero

Learn these in order. You do not need deep machine learning theory to build a working RAG project — the list below is deliberately practical.

1. **Python fundamentals** — functions, classes, working with files and APIs.
2. **What an LLM is and how to call one via an API** — prompts, system messages, responses.
3. **What embeddings are** — the core idea of turning text into vectors (Figure 3 above).
4. **Vector databases and similarity search** — how Chroma or FAISS store and search vectors.
5. **LangChain basics** — document loaders, text splitters, chains (optional but widely used and makes the plumbing easier).
6. **Build the pipeline yourself** — follow Section 4 of this guide end to end with your own files.
7. **(Later) Advanced RAG techniques** — reranking, agentic RAG, evaluation.

7. Curated YouTube Videos

Watch these roughly in order. The first two build genuine intuition before any code; the rest are hands-on implementation walkthroughs.

RAG Explained For Beginners

Concept-first introduction — watch this one first if RAG is completely new to you.

https://www.youtube.com/watch?v=_HQ2H_0Ayy0

RAG Tutorial 2026: Complete Introduction to Retrieval Augmented Generation

Builds a complete RAG application from scratch, beginner-friendly, step by step.

<https://www.youtube.com/watch?v=vAK0iqA6-QI>

Learn RAG From Scratch — Python AI Tutorial from a LangChain Engineer

Implementation-focused course taught by a LangChain engineer, in Python.

<https://www.youtube.com/watch?v=sVcwVQRHlc8>

Retrieval-Augmented Generation (RAG) Chatbot Tutorial for Beginners

Walks through building an actual RAG-powered chatbot end to end.

<https://www.youtube.com/watch?v=mdfUAFH7M9U>

Complete RAG Tutorial 2026 (Free Labs)

A comprehensive crash course with accompanying free hands-on labs.

<https://www.youtube.com/watch?v=vT-DpLv29Q>

Mastering Retrieval-Augmented Generation (RAG) — Playlist

A full playlist covering RAG fundamentals through more advanced techniques.

https://www.youtube.com/playlist?list=PLYX0kXV_HDtFki_NuoMxepcj4kZ42lyzy

Video availability and content can change over time — if a link is ever taken down, search the exact title shown above on YouTube to find its current location or a close equivalent.